

گزارش پیاده‌سازی فراخوان سیستمی getsleepinfo در xv6-riscv

محمد یآوری

تیر ۱۴۰۵

فهرست مطالب

۲	۱ مقدمه
۳	۲ ساختار sleepinfo
۳	۱.۲ تعریف ساختار
۳	۲.۲ فیلدهای ساختار
۳	۱.۲.۲ pid – شناسه فرایند
۴	۲.۲.۲ sleep_ticks – مجموع تیک‌های خواب
۴	۳.۲.۲ sleep_count – تعداد دفعات خواب
۵	۳ پیاده‌سازی
۵	۱.۳ فایل‌های تغییر یافته
۵	۲.۳ ابزار اندازه‌گیری خواب در kernel/proc.c
۶	۳.۳ پیاده‌سازی هسته: sys_getsleepinfo
۸	۴.۳ شمارنده‌های struct proc
۹	۴ رابط فضای کاربر
۹	۱.۴ امضای فراخوان
۹	۲.۴ برنامه sleepinfo
۱۱	۵ نتایج آزمون
۱۱	۱.۵ محیط آزمون
۱۱	۲.۵ آزمون ۱: اجرای پایه (CPUS=1)
۱۲	۳.۵ آزمون ۲: چند پردازنده (CPUS=3)
۱۳	۴.۵ آزمون ۳: اجراهای متوالی
۱۴	۵.۵ آزمون ۴: پس از دستورات سنگین ورودی/خروجی
۱۴	۶.۵ خلاصه آزمون‌ها
۱۶	۶ محدودیت‌ها

فصل ۱

مقدمه

xv6 یک سیستم‌عامل آموزشی ساده است که در MIT توسعه یافته و بر روی معماری RISC-V اجرا می‌شود. هسته این سیستم‌عامل هیچ مکانیزمی برای پرس‌وجو از میزان و تعداد دفعات sleep فرایندها ارائه نمی‌دهد.

در این پروژه، فراخوان سیستمی getsleepinfo به هسته xv6 اضافه شده است. این فراخوان اطلاعات خواب (زمان و تعداد) را برای یک فرایند مشخص یا همه فرایندهای سیستم برمی‌گرداند.

تعریف ۱.۰.۱ فراخوان سیستمی

فراخوان سیستمی رابطی است که فرایندهای فضای کاربر از طریق آن به خدمات هسته دسترسی می‌یابند. در معماری RISC-V، این مکانیزم با دستورالعمل ecall فعال می‌شود.

فصل ۲

ساختار sleepinfo

۱.۲ تعریف ساختار

ساختار sleepinfo در فایل kernel/proc.h تعریف شده است:

قطعه کد ۱.۱.۲ < تعریف struct sleepinfo

```
1 struct sleepinfo {
2     int pid; /* process ID (0 for unused slots) */
3     uint64 sleep_ticks; /* total timer ticks in SLEEPING
   ↪ state */
4     uint64 sleep_count; /* number of times sleep() was called
   ↪ */
5 };
```

۲.۲ فیلدهای ساختار

۱.۲.۲ pid – شناسه فرایند

شناسه عددی فرایند. در پرس‌وجوی همه فرایندها (who == -1)، ورودی‌های متناظر با اسلات‌های آزاد جدول فرایندها مقدار صفر دارند.

۲.۲.۲ sleep_ticks – مجموع تیک‌های خواب

مجموع تیک‌های تایمر است که فرایند در حالت SLEEPING گذرانده است. این مقدار مستقیماً در تابع sleep() در فایل kernel/proc.c اندازه‌گیری می‌شود: درست پیش از sched() مقدار ticks خوانده می‌شود و بلافاصله پس از بازگشت از sched() (یعنی پس از بیدار شدن فرایند) مابه‌التفاوت به این شمارنده افزوده می‌شود.

۳.۲.۲ sleep_count – تعداد دفعات خواب

تعداد دفعاتی که sleep() توسط این فرایند فراخوانی شده است. هر بار که فرایند به صورت داوطلبانه CPU را رها کند – مثلاً برای انتظار I/O دیسک، لوله، یا فراخوان pause() – این شمارنده یک واحد اضافه می‌شود.

فصل ۳

پیاده‌سازی

۱.۳ فایل‌های تغییر یافته

فایل	تغییر
kernel/proc.h	تعریف struct sleepinfo و افزودن sleep_ticks و sleep_count به struct proc
kernel/proc.c	اندازه‌گیری زمان خواب و شمارش تعداد آن در sleep()
kernel/syscall.h	تعریف SYS_getsleepinfo = 22
kernel/syscall.c	ثبت sys_getsleepinfo در جدول توزیع فراخوان‌ها
kernel/sysproc.c	پیاده‌سازی sys_getsleepinfo()
user/user.h	اعلان struct sleepinfo و تابع getsleepinfo()
user/usys.pl	ثبت entry("getsleepinfo")
user/sleepinfo.c	برنامه آزمون فضای کاربر

جدول ۱.۳: فایل‌های تغییر یافته در پیاده‌سازی getsleepinfo

۲.۳ ابزار اندازه‌گیری خواب در kernel/proc.c

تابع sleep() در فایل kernel/proc.c برای ردیابی آمار خواب دستکاری شده است:

قطعه کد ۱.۲.۳ ◀ اندازه‌گیری خواب در sleep()

```
1 // Go to sleep.
2 p->sleep_count++;
```

```

3  p->chan = chan;
4  p->state = SLEEPING;
5
6  uint sleep_start = ticks; // best-effort unsynchronized read
7  sched();
8  p->sleep_ticks += ticks - sleep_start;

```

متغیر ticks بدون قفل‌گذاری خوانده می‌شود. این همان رویکردی است که xv6 برای سایر شمارنده‌های آماری فرایند به کار می‌برد. خواندن همروند از ticks در معماری RISC-V برای مقادیر ۳۲ بیتی اتمیک است و در عمل خطای ناچیزی ایجاد می‌کند.

۳.۳ پیاده‌سازی هسته: sys_getsleepinfo

تابع sys_getsleepinfo() در فایل kernel/sysproc.c پیاده‌سازی شده است. آرگومان اول (who) حالت کار را مشخص می‌کند:

قطعه کد ۱.۳.۳ < پیاده‌سازی sys_getsleepinfo

```

1  uint64
2  sys_getsleepinfo(void)
3  {
4      int who;
5      uint64 addr;
6      argint(0, &who);
7      argaddr(1, &addr);
8
9      struct proc *p = myproc();
10
11     if (who == -1) {
12         struct sleepinfo buf[NPROC];
13         for (int i = 0; i < NPROC; i++) {
14             struct proc *pp = &proc[i];
15             acquire(&pp->lock);
16             buf[i].pid = (pp->state != UNUSED) ? pp->pid :
                0;

```

```
17     buf[i].sleep_ticks = pp->sleep_ticks;
18     buf[i].sleep_count = pp->sleep_count;
19     release(&pp->lock);
20 }
21 if (copyout(p->pagetable, addr, (char *)buf,
22     ↳ sizeof(buf)) < 0)
23     return -1;
24 return 0;
25 }
26 struct proc *target = 0;
27 if (who == 0) {
28     target = p;
29 } else {
30     for (struct proc *pp = proc; pp < &proc[NPROC]; pp++) {
31         acquire(&pp->lock);
32         if (pp->state != UNUSED && pp->pid == who) {
33             target = pp;
34             release(&pp->lock);
35             break;
36         }
37         release(&pp->lock);
38     }
39     if (target == 0)
40         return -1;
41 }
42
43 struct sleepinfo si;
44 acquire(&target->lock);
45 si.pid = target->pid;
46 si.sleep_ticks = target->sleep_ticks;
47 si.sleep_count = target->sleep_count;
48 release(&target->lock);
49
```



```

50     if (copyout(p->pagetable, addr, (char *)&si, sizeof(si)) <
        ↪ 0)
51         return -1;
52     return 0;
53 }

```

۴.۳ شمارنده‌های struct proc

دو فیلد جدید به struct proc در kernel/proc.h اضافه شده‌اند:

قطعه کد ۱.۴.۳: فیلدهای جدید در struct proc

```

1  uint64 sleep_ticks; // total ticks spent in SLEEPING state
2  uint64 sleep_count; // number of times sleep() was called

```

این فیلدها در بخش خصوصی struct proc (که نیازی به نگه‌داشتن p->lock ندارند) قرار گرفته‌اند، چون فقط توسط خود فرایند نوشته می‌شوند (از داخل sleep() که p->lock را نگه داشته است).

فصل ۴

رابط فضای کاربر

۱.۴ امضای فراخوان

قطعه کد ۱.۱.۴ < امضای getsleepinfo

```
1 int getsleepinfo(int who, struct sleepinfo *buf);
```

مقدار who	معنا
-----------	------

0	فرایند جاری – یک ورودی sleepinfo در buf
> 0	فرایند با pid == who – یک ورودی در buf، یا -1 اگر پیدا نشد
-1	همه فرایندها – NPROC ورودی در buf (اسلات‌های آزاد: pid = 0)

جدول ۱.۴: مقادیر آرگومان who

۲.۴ برنامه sleepinfo

برنامه sleepinfo (فایل user/sleepinfo.c) موارد زیر را آزمون می‌کند:

قطعه کد ۱.۲.۴ < نمونه استفاده از getsleepinfo

```
1 /* current process */
2 struct sleepinfo si;
3 getsleepinfo(0, &si);
```

```
4
5  /* after pause(5) */
6  pause(5);
7  getsleepinfo(0, &si);
8
9  /* by explicit pid */
10 getsleepinfo(getpid(), &si);
11
12 /* invalid pid – should return -1 */
13 getsleepinfo(99999, &si);
14
15 /* all processes */
16 struct sleepinfo all[NPROC];
17 getsleepinfo(-1, all);
```

فصل ۵

نتایج آزمون

۱.۵ محیط آزمون

همه آزمون‌ها در QEMU انجام شده‌اند. مگر اینکه خلاف آن ذکر شود، CPUS=1 استفاده می‌شود. دستور `make qemu CPUS=1` برای راه‌اندازی xv6 به کار می‌رود.

۲.۵ آزمون ۱: اجرای پایه (CPUS=1)

بلافاصله پس از بوت، اجرای `sleepinfo` به عنوان اولین دستور:

قطعه کد ۱.۲.۵ > آزمون ۱ – بوت تازه، CPUS=1

```
1 $ sleepinfo
2 self (before sleep): pid=3  sleep_ticks=1  sleep_count=5
3 self (after  sleep): pid=3  sleep_ticks=6  sleep_count=10
4 self (by pid)       : pid=3  sleep_ticks=6  sleep_count=10
5 getsleepinfo(99999): correctly returned -1
6
7 all processes:
8   pid=1  sleep_ticks=0  sleep_count=23
9   pid=2  sleep_ticks=45  sleep_count=11
10  pid=3  sleep_ticks=6  sleep_count=10
```

مثال ۲.۲.۵ < تحلیل آزمون ۱

- sleep_ticks از ۱ به ۶ رسید - pause(5) دقیقاً ۵ تیک اضافه کرد.
- sleep_count از ۵ به ۱۰ رسید - ورودی/خروجی کنسول چندین فراخوانی sleep() هنگام تجزیه دستور ایجاد می‌کند؛ pause(5) فراخوانی‌های بیشتری اضافه می‌کند.
- پرس‌وجو با PID صریح مقادیر یکسان با 0 == who برمی‌گرداند.
- PID نامعتبر (99999) به درستی 1- برمی‌گرداند.
- پرس‌وجوی همه فرایندها (pid= init)، (pid= sh)، (pid= sleepinfo) و (pid= ۳) را نشان می‌دهد.

۳.۵ آزمون ۲: چند پردازنده (CPUS=3)

همان آزمون با CPUS=3 برای بررسی صحت در پیکربندی چند پردازنده‌ای:

قطعه کد ۱.۳.۵ < آزمون ۲ - CPUS=3

```

1 $ sleepinfo
2 self (before sleep): pid=3 sleep_ticks=0 sleep_count=5
3 self (after sleep): pid=3 sleep_ticks=5 sleep_count=10
4 self (by pid)      : pid=3 sleep_ticks=5 sleep_count=10
5 getsleepinfo(99999): correctly returned -1
6
7 all processes:
8   pid=1 sleep_ticks=0 sleep_count=23
9   pid=2 sleep_ticks=44 sleep_count=11
10  pid=3 sleep_ticks=5 sleep_count=10

```

مثال ۲.۳.۵ < تحلیل آزمون ۲

- اختلاف sleep_ticks همچنان ۵ است و دقت pause(5) را در محیط چند پردازنده تأیید می‌کند.
- sleep_ticks اولیه ۰ است (در مقابل ۱ در CPUS=1) - تفاوت جزئی ناشی از زمان‌بندی چند هسته‌ای.
- sh مقدار ۴۴ تیک خواب انباشت کرد (در مقابل ۴۵ در CPUS=1) - مطابق با خطای ± 1 تیک توضیح داده شده در فصل محدودیت‌ها.

۴.۵ آزمون ۳: اجراهای متوالی

اجرای دوبار متوالی sleepinfo برای مشاهده اینکه هر اجرا فرایند مستقلی با شمارنده‌های خود است:

قطعه کد ۱.۴.۵ - آزمون ۳ - دو اجرای متوالی

```

1 $ sleepinfo
2 self (before sleep): pid=3  sleep_ticks=0  sleep_count=5
3 self (after  sleep): pid=3  sleep_ticks=5  sleep_count=10
4 self (by pid)      : pid=3  sleep_ticks=5  sleep_count=10
5 getsleepinfo(99999): correctly returned -1
6
7 all processes:
8   pid=1  sleep_ticks=0  sleep_count=23
9   pid=2  sleep_ticks=45  sleep_count=11
10  pid=3  sleep_ticks=5  sleep_count=10
11 $ sleepinfo
12 self (before sleep): pid=4  sleep_ticks=5  sleep_count=10
13 self (after  sleep): pid=4  sleep_ticks=10  sleep_count=15
14 self (by pid)      : pid=4  sleep_ticks=10  sleep_count=15
15 getsleepinfo(99999): correctly returned -1
16
17 all processes:
18   pid=1  sleep_ticks=0  sleep_count=23
19   pid=2  sleep_ticks=194  sleep_count=14
20   pid=4  sleep_ticks=10  sleep_count=15

```

مثال ۲.۴.۵ - تحلیل آزمون ۳

- اجرای دوم (pid=4 گرفت pid=3) پس از خروج اجرای اول آزاد شد).
- xv6 ساختار struct proc را در freeproc() صفر می‌کند، اما اسلات‌های جدول فرایندها را بازاستفاده می‌کند. مقادیر اولیه sleep_ticks=5 و sleep_count=10 در اجرای دوم نشان می‌دهد اسلات با شمارنده‌های باقی‌مانده بازاستفاده شده (allocproc) در xv6 کل ساختار را صفر نمی‌کند - فقط فیلدهای منتخب).
- sleep_ticks پوستره (sh) از ۴۵ به ۱۹۴ رسید - پوستره در زمان بیکاری بین دو دستور خوابیده بود.
- sleep_count پوستره از ۱۱ به ۱۴ رسید - سه فراخوانی خواب اضافی برای خواندن کنسول.

۵.۵ آزمون ۴: پس از دستورات سنگین ورودی/خروجی

اجرای grep و cat README قبل از sleepinfo برای مشاهده تأثیر ورودی/خروجی دیسک بر شمارنده‌های خواب:

قطعه کد ۱.۵.۵ < آزمون ۴ – پس از cat README + grep

```

1 $ grep hello README
2 $ cat README
3 xv6 is a re-implementation of Dennis Ritchie's and Ken
  ↪ Thompson's Unix
4 Version 6 (v6). ...
5 $ sleepinfo
6 self (before sleep): pid=5  sleep_ticks=5  sleep_count=2380
7 self (after sleep): pid=5  sleep_ticks=10  sleep_count=2385
8 self (by pid)      : pid=5  sleep_ticks=10  sleep_count=2385
9 getsleepinfo(99999): correctly returned -1
10
11 all processes:
12   pid=1  sleep_ticks=0  sleep_count=23
13   pid=2  sleep_ticks=94  sleep_count=17
14   pid=5  sleep_ticks=10  sleep_count=2385

```

مثال ۲.۵.۵ < تحلیل آزمون ۴

- sleep_count قبل از خواب صریح ۲۳۸۰ است – به شدت بالاتر از مقدار ۵ در آزمون ۱. دستورات grep و cat هزاران فراخوانی (sleep() ناشی از خواندن دیسک (virtio block I/O) ایجاد کرده‌اند و اسلات جدول فرایندها با شمارنده‌های انباشته بازاستفاده شده.
- اختلاف sleep_ticks همچنان دقیقاً ۵ است و تأیید می‌کند اندازه‌گیری تیک مستقل از تعداد فراخوانی است.
- pid=5 (نه ۳) چون شناسه‌های ۳ و ۴ توسط فرایندهای grep و cat مصرف شدند.
- sh مقدار ۹۴ تیک خواب انباشت کرد (در مقابل ۴۵ در آزمون ۱) – زمان بیکاری بیشتر بین دستورات متعدد.

۶.۵ خلاصه آزمون‌ها

آزمون	پیکربندی	$\Delta ticks$	موفق	مشاهده کلیدی
۱. پایه	CPUS=1	۵	بله	خط پایه؛ تمام حالات پرس‌وجو صحیح
۲. چند پردازنده	CPUS=3	۵	بله	تفاوت ± 1 تیک، سازگار
۳. متوالی	CPUS=1	۵	بله	بازاستفاده اسلات؛ آمار پوسته انباشته
۴. پس از I/O	CPUS=1	۵	بله	ورودی/خروجی دیسک sleep_count را افزایش می‌دهد

جدول ۱.۵: خلاصه تمام اجراهای آزمون. $\Delta ticks$ افزایش sleep_ticks ناشی از (5) pause است.

فصل ۶

محدودیت‌ها

خواندن **ticks** بدون قفل. متغیر **ticks** بدون **tickslock** خوانده می‌شود. اگر **tick-slock** درون **sleep()** در حالی که **p->lock** نگه داشته می‌شود گرفته شود، همه تابع‌هایی که **tickslock** را نگه می‌دارند و سپس **sleep()** را فراخوانی می‌کنند (مثل **sys_pause**) باید از نظر ترتیب قفل‌گذاری بررسی شوند. خواندن غیرهمگام‌شده بهترین رویکرد عملی است.

خطای اندازه‌گیری در چند پردازنده. در حالت $CPU > 1$ ، متغیر **ticks** فقط توسط CPU شماره صفر افزایش می‌یابد (**clockintr** در **kernel/trap.c**). پردازنده‌های دیگر همان مقدار را می‌خوانند اما این مقدار ممکن است کمی کهنه باشد. برای **xv6** با **CPU=3**، این خطا معمولاً کمتر از یک تیک است.

تفاوت با **sleep** هسته و **sleeplock**. این آمار فقط **sleep()** کرنل را ردیابی می‌کند که حالت فرایند را به **SLEEPING** تغییر می‌دهد. **sleeplock** نیز از همان مکانیزم استفاده می‌کند و بنابراین در **sleep_count** شمرده می‌شود.